SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT -1 CO 1

NAME : Kristen Lawrence

REG NO : 192520028

COURSE CODE : CSA0825 PYTHON PROGRAMMING

GITHUB LINK : https://github.com/kris10-exe/PYTHON---CSA0825---192520028---KRISTEN-CSA0825-PYTHON

Task 1 - Milk Payment Data Analysis

Aim: To identify the inputs, outputs and rules required to calculate milk payments.

Algorithm:

1. Start.
2. Define the base rate, standard fat, standard SNF and step values.
3. Store the farmer details, milk quantity, fat and SNF values.
4. Calculate the fat and SNF step differences.
5. Calculate the rate per litre.
6. Calculate the payment for each farmer.
7. Add each payment to the total.
8. Display the rate, payment and total payment.
9. Stop.

PROGRAM CODE :

```
BASE_RATE = 30.00

entries = [
    ("F001", 10, 3.5, 8.5),
    ("F002", 8.5, 4.2, 8.7)
]

total = 0

for farmer, litres, fat, snf in entries:
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = BASE_RATE + fat_steps * 0.40
    rate += snf_steps * 0.30

    payment = round(litres * rate, 2)
    total += payment

    print(farmer, "Rate:", round(rate, 2))
    print("Payment:", payment)

print("Total:", round(total, 2))
```

OUTPUT :

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/T.Jaya Krishna Reddy/AppData/Local/Programs/Python/Python314
/task 1.py
F001 Rate: 30.0
Payment: 300.0
F002 Rate: 33.4
Payment: 283.9
Total: 583.9
```

Task 2 — Data Types, Operators and Decimal

Aim: To demonstrate Python data types, arithmetic operators and Decimal calculations.

Algorithm:

1. Start.
2. Import the Decimal class.
3. Create variables of different data types.
4. Display the types of the variables.
5. Perform arithmetic operations using /, // and %.
6. Compare floating-point arithmetic with Decimal arithmetic.
7. Display the results.
8. Stop.

PROGRAM CODE :

```
from decimal import Decimal

print(type(10))
print(type(10.5))
print(type("Hello"))
print(type(True))
print(type([10, 20]))

print(7 / 2)
print(7 // 2)
print(7 % 2)
print(-7 // 2)

print(0.1 + 0.2 == 0.3)

a = Decimal("0.1")
b = Decimal("0.2")

print(a + b)
print(a + b == Decimal("0.3"))

payments = ["10.10", "20.20", "30.30"]

total = Decimal("0.00")

for payment in payments:
    total += Decimal(payment)

print("Total:", total)
```

OUT PUT :

```
/TASK 2.py
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'list'>
3.5
3
1
-4
False
0.3
True
Total: 60.60
>
```

Task 3 — Variables, Constants and Assignment

Aim: To demonstrate variables, constants, dynamic typing and assignment operations in Python.

Algorithm:

1. Start.
2. Define the base rate and step values as constants.
3. Create a variable and assign an integer value.
4. Display the variable and its type.
5. Assign a string value to the same variable.
6. Swap the values of two variables.
7. Create a dictionary containing the rate details.
8. Display the results.
9. Stop.

PROGRAM CODE :

```
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30

value = 10
print(value, type(value))

value = "Milk"
print(value, type(value))

a = 10
b = 20

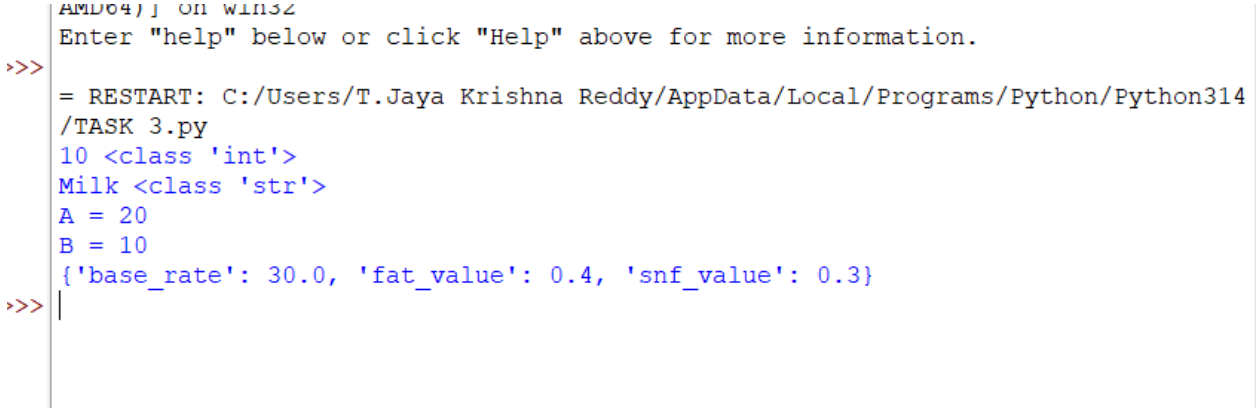
a, b = b, a

print("A =", a)
print("B =", b)

rate_card = {
    "base_rate": BASE_RATE,
    "fat_value": FAT_STEP_VALUE,
    "snf_value": SNF_STEP_VALUE
}

print(rate_card)
```

OUTPUT :



Task 4 — Expressions and Rounding

Aim: To calculate the milk rate and payment using arithmetic expressions and rounding operations.

Algorithm:

1. Start.
2. Read the milk quantity, fat percentage and SNF percentage.
3. Calculate the fat step difference.
4. Calculate the SNF step difference.
5. Calculate the rate per litre using the base rate and step values.
6. Calculate the total payment by multiplying quantity by rate.
7. Apply rounding where required.
8. Display the step differences, rate and payment.
9. Stop.

PROGRAM CODE :

```
import math

fat = 4.2
snf = 8.7
litres = 8.5

fat_steps = round((fat - 3.5) / 0.1)
snf_steps = round((snf - 8.5) / 0.1)

rate = 30 + fat_steps * 0.40
rate += snf_steps * 0.30

payment = round(litres * rate, 2)

print("Fat steps:", fat_steps)
print("SNF steps:", snf_steps)
print("Rate:", round(rate, 2))
print("Payment:", payment)

print("Floor:", math.floor(-7 / 2))
print("Integer:", int(-7 / 2))
print("Float:", 0.1 + 0.2)
```

OUTPUT :

```
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>
= RESTART: C:/Users/T.Jaya Krishna Reddy/AppData/Local/Programs/Python/Python314
/TASK 4.py
Fat steps: 7
SNF steps: 2
Rate: 33.4
Payment: 283.9
Floor: -4
Integer: -3
Float: 0.30000000000000004
>>
```

Task 5 — User Input and Validation

Aim: To accept milk details from the user, validate the inputs and calculate the payment.

Algorithm:

1. Start.
2. Read litres, fat percentage and SNF percentage.
3. Check whether the milk quantity is negative.
4. Check whether fat is between 2.0 and 7.0.
5. Check whether SNF is non-negative.
6. If any input is invalid, display an error message.
7. Otherwise, calculate the fat and SNF step differences.
8. Calculate the rate and total payment.
9. Display the rate and payment.
10. Stop.

PROGRAM CODE :

```
litres = float(input("Enter litres: "))
fat = float(input("Enter fat: "))
snf = float(input("Enter SNF: "))

if litres < 0:
    print("Invalid litres")
elif fat < 2 or fat > 7:
    print("Invalid fat")
elif snf < 0:
    print("Invalid SNF")
else:
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = 30 + fat_steps * 0.40
    rate += snf_steps * 0.30

    payment = round(litres * rate, 2)

    print(f"Rate: Rs. {rate:.2f}")
    print(f"Payment: Rs. {payment:.2f}")
```

OUTPUT :

```
>>
= RESTART: C:/Users/T.Jaya Krishna Reddy/AppData/Local/Programs/Python/Python314
/TASK 5.py
Enter litres: 12
Enter fat: 40
Enter SNF: 45
Invalid fat
>>
```

Task 6 — Functions and Return Values

Aim: To develop reusable functions for calculating the milk rate, individual payment and ten-day total.

Algorithm:

1. Start.
2. Define a function to calculate the rate per litre.
3. Define a function to calculate the payment.
4. Define a function to calculate the ten-day total.
5. Store the milk entries in a list.
6. Call the payment function for each entry.
7. Add the individual payments to the total.
8. Display the individual payments and ten-day total.
9. Stop.

PROGRAM CODE :

```
File  Edit  Format  Run  Options  Window  Help
def rate_per_litre(fat, snf):
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = 30 + fat_steps * 0.40
    rate += snf_steps * 0.30

    return round(rate, 2)


def payout(litres, fat, snf):
    return round(
        litres * rate_per_litre(fat, snf), 2
    )


def ten_day_total(entries):
    total = 0

    for litres, fat, snf in entries:
        total += payout(litres, fat, snf)

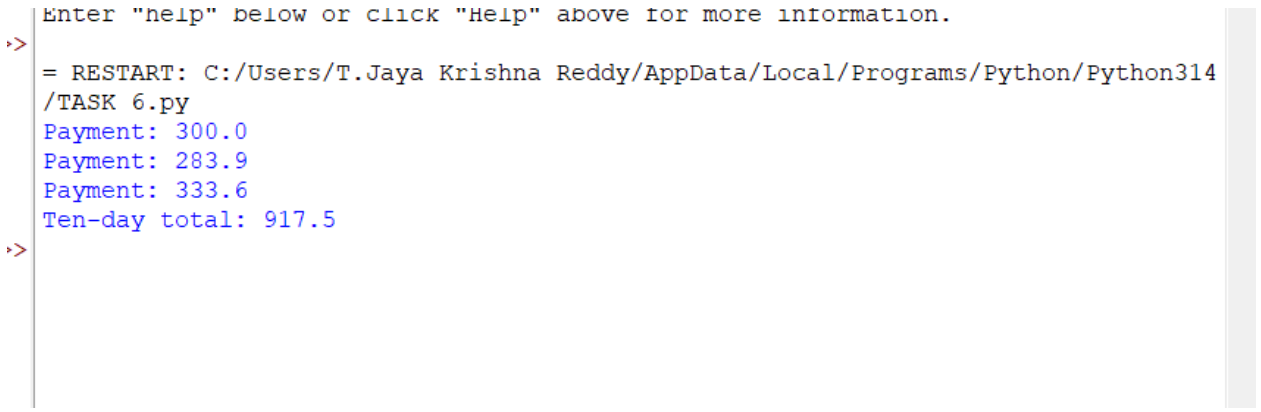
    return round(total, 2)


entries = [
    (10, 3.5, 8.5),
    (8.5, 4.2, 8.7),
    (12, 3.1, 8.3)
]

for entry in entries:
    print("Payment:", payout(*entry))

print("Ten-day total:", ten_day_total(entries))
```

OUTPUT :



Task 7 — Creating and Importing a Module

Aim: To create a Python module containing milk-rate functions and import it into another program.

Algorithm:

1. Start.
2. Create a file named milkrate.py.
3. Define the rate calculation function.
4. Define the payment calculation function.
5. Save the module.
6. Create another file named test_milkrate.py.
7. Import the required functions from milkrate.
8. Call the imported functions with sample values.
9. Display the rate and payment.
10. Stop.

PRROGRAM OUTPUT :

```
BASE_RATE = 30.00


def rate_per_litre(fat, snf):
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = BASE_RATE + fat_steps * 0.40
    rate += snf_steps * 0.30

    return round(rate, 2)


def payout(litres, fat, snf):
    return round(
        litres * rate_per_litre(fat, snf), 2
    )


if __name__ == "__main__":
    print(rate_per_litre(4.2, 8.7))
```

OUTPUT :

```
>>
= RESTART: C:/Users/T.Jaya Krishna Reddy/AppData/Local/P
/TASK 7.py
33.4
>>
```

Task 8 — Algorithm, Pseudocode and Hand Tracing \

Aim: To implement and trace the milk-payment algorithm and calculate the running total.

Algorithm:

1. Start.
2. Initialise the total payment to zero.
3. Read or retrieve the milk entries.
4. Validate each entry.
5. Calculate the fat and SNF step differences.
6. Calculate the rate per litre.
7. Calculate the individual payment.
8. Add the payment to the running total.
9. Repeat until all entries are processed.
10. Stop.

PROGRAM CODE :

```
entries = [
    (10, 3.5, 8.5),
    (8.5, 4.2, 8.7),
    (12, 3.1, 8.3)
]

total = 0

for litres, fat, snf in entries:
    if litres < 0 or fat < 2 or fat > 7 or snf < 0:
        print("Invalid entry")
        continue

    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = 30 + fat_steps * 0.40
    rate += snf_steps * 0.30

    payment = round(litres * rate, 2)
    total += payment

    print("Rate:", round(rate, 2))
    print("Payment:", payment)

print("Total:", round(total, 2))
```

OUTPUT :

```
>>
= RESTART: C:/Users/T.Jaya Krishna Reddy/AppData/Local/Programs/Python/Python314
/TASK 8.py
Rate: 30.0
Payment: 300.0
Rate: 33.4
Payment: 283.9
Rate: 27.8
Payment: 333.6
Total: 917.5
>>
```

Task 9 — Complete Calculator and Automated Testing

Aim: To develop a milk-payment calculator and test normal, boundary and invalid inputs.

Algorithm:

1. Start.
2. Define the rate calculation function.
3. Validate fat, SNF and milk quantity.
4. Calculate the rate and payment for valid inputs.
5. Write test cases for normal values.
6. Write test cases for minimum and maximum fat values.
7. Test invalid values using exception handling.
8. Use assertions to compare actual and expected results.
9. Display the test results.
10. Stop.

PROGRAM CODE :

```
def rate_per_litre(fat, snf):
    if not 2 <= fat <= 7:
        raise ValueError("Invalid fat")

    if snf < 0:
        raise ValueError("Invalid SNF")

    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = 30 + fat_steps * 0.40
    rate += snf_steps * 0.30

    return round(rate, 2)


def payout(litres, fat, snf):
    if litres < 0:
        raise ValueError("Invalid litres")

    return round(litres * rate_per_litre(fat, snf), 2)


assert payout(10, 3.5, 8.5) == 300.00
assert payout(8.5, 4.2, 8.7) == 283.90
assert payout(12, 3.1, 8.3) == 333.60

assert rate_per_litre(2.0, 8.5) == 24.00
assert rate_per_litre(7.0, 8.5) == 44.00

print("All tests passed")
```

OUT PUT :

```
Enter "help" below or click "Help" above for more information.
>>
= RESTART: C:/Users/T.Jaya Krishna Reddy/AppData/Local/Programs/Python/Python31
/TASK 9.py
All tests passed
>>
```

Task 10 — Performance Comparison: Float vs Decimal
from decimal import Decimal

Aim: To compare the execution time and numerical results of float and Decimal calculations for multiple payments.

Algorithm:

1. Start.
2. Import Decimal and perf_counter.
3. Define a function to calculate totals using float.
4. Define another function to calculate totals using Decimal.
5. Create payment lists of different sizes.
6. Record the start time for float calculations.
7. Calculate the float total and elapsed time.
8. Repeat using Decimal.
9. Display both totals and execution times.
10. Stop.

PRROGRAM CODE :

```
from decimal import Decimal
from time import perf_counter


def total_float(payments):
    total = 0.0

    for payment in payments:
        total += float(payment)

    return total


def total_decimal(payments):
    total = Decimal("0.00")

    for payment in payments:
        total += Decimal(payment)

    return total


for n in (300, 6000, 150000):
    payments = ["33.40"] * n

    start = perf_counter()
    f_total = total_float(payments)
    f_time = perf_counter() - start

    start = perf_counter()
    d_total = total_decimal(payments)
    d_time = perf_counter() - start

    print("Payments:", n)
    print("Float total:", f_total)
    print("Float time:", f_time)

    print("Decimal total:", d_total)
    print("Decimal time:", d_time)
    print("--------------------")
```

OUTPUT :

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 b
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>
= RESTART: C:/Users/T.Jaya Krishna Reddy/AppData/Local/Programs/Python/Pytho
/TASK 10.py
Payments: 300
Float total: 10019.999999999944
Float time: 7.6299998909235e-05
Decimal total: 10020.00
Decimal time: 0.00019940000493079424
--------------------
Payments: 6000
Float total: 200399.99999997852
Float time: 0.001381900001433678
Decimal total: 200400.00
Decimal time: 0.0032315999997081235
--------------------
Payments: 150000
Float total: 5010000.000000843
Float time: 0.036619999998947605
Decimal total: 5010000.00
Decimal time: 0.10489229999075178
--------------------
```

Task 11 — Recommended Design from decimal import Demical

Aim: To develop a reliable milk-payment calculator using Decimal arithmetic, validation and reusable functions.

Algorithm:

1. Start.
2. Define named constants for the milk rate.
3. Create a function to calculate the rate using Decimal.
4. Validate the fat, SNF and quantity values.
5. Calculate the rate per litre.
6. Calculate the payment using Decimal arithmetic.
7. Round the payment to two decimal places.
8. Display the payment.
9. Stop.

PRROGRRAM CODE :

```
from decimal import Decimal

BASE_RATE = Decimal("30.00")
FAT_VALUE = Decimal("0.40")
SNF_VALUE = Decimal("0.30")


def rate_per_litre(fat, snf):
    fat = Decimal(fat)
    snf = Decimal(snf)

    if not Decimal("2.0") <= fat <= Decimal("7.0"):
        raise ValueError("Invalid fat")

    if snf < 0:
        raise ValueError("Invalid SNF")

    fat_steps = ((fat - Decimal("3.5")) /
                 Decimal("0.1")).quantize(Decimal("1"))

    snf_steps = ((snf - Decimal("8.5")) /
                 Decimal("0.1")).quantize(Decimal("1"))

    rate = BASE_RATE + fat_steps * FAT_VALUE
    rate += snf_steps * SNF_VALUE

    return rate.quantize(Decimal("0.01"))


def payout(litres, fat, snf):
    litres = Decimal(litres)

    if litres < 0:
        raise ValueError("Invalid litres")

    payment = litres * rate_per_litre(fat, snf)
    return payment.quantize(Decimal("0.01"))


print("Payment:", payout("8.5", "4.2", "8.7"))
```

OUTPUT :

```
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/T.Jaya Krishna Reddy/AppData/Local/Programs/Python/Python314
/TASK 11.py
Payment: 283.90
>>>
```

Task 12 — Seasonal Rates and Bonus Calculation

Aim: To calculate milk payments using seasonal rate cards, two-decimal readings and a ten-day supply bonus.

Algorithm:

1. Start.
2. Define the standard fat and SNF values.
3. Define the regular and seasonal rate cards.
4. Read or retrieve the ten-day milk entries.
5. Select the required season.
6. Validate the milk quantity, fat and SNF values.
7. Calculate the rate for each entry using Decimal.
8. Calculate the individual payments and base total.
9. Calculate the total milk quantity supplied.
10. STOP

PROGRAM CODE :

```
from decimal import Decimal, ROUND_HALF_UP

STEP = Decimal("0.10")
STANDARD_FAT = Decimal("3.50")
STANDARD_SNF = Decimal("8.50")

RATE_CARDS = {
    "regular": {
        "base": Decimal("30.00"),
        "fat": Decimal("0.40")
    },
    "april_june": {
        "base": Decimal("32.00"),
        "fat": Decimal("0.45")
    }
}

SNF_VALUE = Decimal("0.30")
BONUS_LIMIT = Decimal("500")
BONUS_RATE = Decimal("0.50")


def round_steps(value):
    return (value / STEP).quantize(
        Decimal("1"),
        rounding=ROUND_HALF_UP
    )


def rate_per_litre(fat, snf, season="regular"):
    fat = Decimal(fat)
    snf = Decimal(snf)

    if season not in RATE_CARDS:
        raise ValueError("Invalid season")

    if not Decimal("2.00") <= fat <= Decimal("7.00"):
        raise ValueError("Invalid fat")

    if snf < 0:
        raise ValueError("Invalid SNF")

    card = RATE_CARDS[season]

    fat_steps = round_steps(fat - STANDARD_FAT)
    snf_steps = round_steps(snf - STANDARD_SNF)

    rate = card["base"] + fat_steps * card["fat"]
    rate += snf_steps * SNF_VALUE

    return rate.quantize(Decimal("0.01"))
```

```
def payout(litres, fat, snf, season="regular"):
    litres = Decimal(litres)

    if litres < 0:
        raise ValueError("Invalid litres")

    rate = rate_per_litre(fat, snf, season)

    return (litres * rate).quantize(
        Decimal("0.01"),
        rounding=ROUND_HALF_UP
    )


def ten_day_payment(entries, season="regular"):
    total_litres = sum(
        (Decimal(e[0]) for e in entries),
        Decimal("0")
    )

    base_total = sum(
        (payout(*e, season) for e in entries),
        Decimal("0.00")
    )

    bonus = Decimal("0.00")

    if total_litres > BONUS_LIMIT:
        bonus = (total_litres * BONUS_RATE).quantize(
            Decimal("0.01")
        )

    final_total = base_total + bonus

    return base_total, bonus, final_total


entries = [
    ("250.00", "4.25", "8.65"),
    ("260.50", "3.55", "8.50")
]

base, bonus, final = ten_day_payment(
    entries, "april_june"
)

print("Base payment:", base)
print("Bonus:", bonus)
print("Final payment:", final)
```

OUT PUT :

```
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/T.Jaya Krishna Reddy/AppData/Local/Programs/Python/Python314
/TASK 12.py
Base payment: 17503.23
Bonus: 255.25
Final payment: 17758.48
>>>
```